

EngTrace template certification — guide for expert reviewers

September 2026

1. What you are certifying

EngTrace is a benchmark of engineering problems. Each problem is produced by a **template**: a short Python function that samples realistic parameters, computes the answer step by step, and writes out a question and a worked solution. A template that is physically wrong, mathematically wrong, or unclear produces hundreds of wrong or confusing problems, so every template is certified by three experts of its own branch before it is used.

You will review the 30 templates of your branch, plus a few extra items described in section 5. For each one you will:

1. **solve one generated instance yourself** and enter your answer before you see ours; 2. read the worked solution and, if you wish, the source code; 3. score the template on three dimensions and decide **Approve** or **Reject**.

Budget about 5 to 8 minutes per template. There is no benefit in going faster: the certification is only worth something if each template was actually checked.

2. The hand check

For each template the app first shows you **one generated question, without its solution**. Work it as you would set it for a student: on paper, a calculator or a spreadsheet, whatever you normally use. Enter the final numerical answer(s) in the box, with the unit. If the question asks for several quantities, enter each. If the answer is not a number (a classification, an expression), enter it in words.

Then press **Show solution**. The app tells you whether your value matches ours within 1%. **A mismatch is not a verdict on you or on the template**: it is a signal to look closely at the solution and find which side is right. If the template is wrong, reject it and say where. If you slipped, say so in the notes and carry on.

The hand check is recorded with your verdict. It is what turns "three experts approved this" into evidence.

3. The three scores

Score each dimension from 1 to 5 after reading the solution and, where useful, the code.

- **Physical plausibility**. Does the scenario respect the physics and engineering of the domain? Are the sampled values realistic for the setting: material properties, sizes, loads, temperatures, rates? A correct formula applied to an impossible object still fails here.
- **Mathematical correctness**. Are the governing equations right? Does every printed step follow from the numbers printed before it? Is the final answer right, with the right unit and a sensible precision?
- **Pedagogical clarity**. Is the question unambiguous and solvable from what it states? Is the solution readable and complete: no missing step, no leftover placeholder, no formatting artefact such as `55.000000000000001`?

Use the whole scale. **5** means you would set this problem as it stands; **4** means minor blemishes that do not affect the answer; **3** means a real weakness a student would trip on; **2** means a substantive error; **1** means the template should not exist in this form.

4. Approve or Reject

Approve means: a student who solves this problem correctly gets the printed answer, and nothing in it would mislead them.

Reject if any of the following holds:

- the physics, a governing equation, a constant, a unit conversion or a sign is wrong;
- a printed step does not follow from the printed numbers before it, beyond ordinary rounding of the last digit;
- the final answer is wrong, or has the wrong unit;
- the scenario is physically implausible (a rubber rod carrying 300 kN, a "bronze wooden block");
- the question cannot be solved from what it states.

Do **not** reject for style you would have written differently, for a different but valid solution path, or for a last-digit rounding difference. Score those under clarity and approve.

A rejection requires a note. Write what is wrong and where, in one or two sentences, and tick the defect type. Your note goes straight to the author, so be specific: "Step 3 uses $g = 9.18$ " is useful; "physics looks off" is not.

5. Quality-control items

A small number of the items in your queue are **deliberately defective**: real templates of your branch into which one known error has been written. They are not marked, and they look like any other template. They exist so that the certification can report a detection rate rather than only an approval rate. You do not need to hunt for them; review every item the same way and they take care of themselves.

6. What the app shows

- **Question** for the hand check, then **Solution** with the printed steps and answer.
- **Source code**, in a tab, for those who want to see how the parameters are sampled and the answer computed. You do not need to read Python to certify a template; the rendered instances are the object of review. The code is there when a number looks odd and you want to know where it came from.
- **Another instance**: the app holds five generated instances of each template. Cycling through them shows how the parameters vary; a defect that appears in one instance and not another is worth a note.

7. Practicalities

- Work saves as you submit each template. You can close the app and continue later; your queue resumes where you stopped.
- Do the templates in the order the app gives them; the order is deliberately shuffled.
- Do not discuss templates with the other reviewers of your branch until all three of you have submitted. Independent verdicts are the point.

- When you finish, send back the file the app names on its last screen. If you prefer to work from a file instead of the app, the workbook route is described in the HOW-TO-RUN note that came with this guide.
- Questions about the task go to the coordinator, not to the other reviewers.

Thank you. The benchmark's claim to be verifiable rests on this review being real, and the record will show that it was.